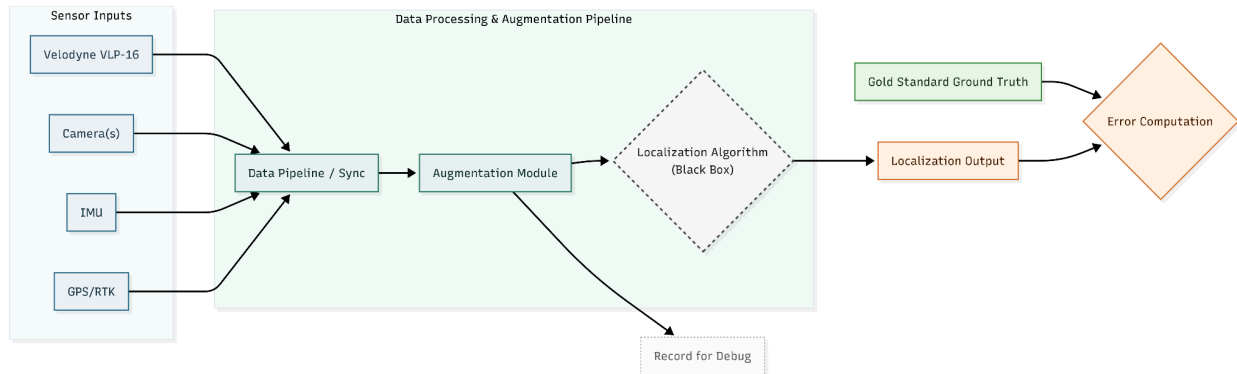


Generalized Perturbation-Based Localization Robustness Evaluation Framework

This document presents a strategy for evaluating the robustness of localization algorithms under realistic real-world operating conditions. The approach treats the localization system as a black box and systematically injects configurable sensor degradations into recorded datasets to quantify their impact on localization accuracy, stability, and reliability by comparison against a Gold Standard trajectory.



Sensor Inputs

- **LiDAR (Velodyne VLP-16):** Velodyne VLP-16 3D LiDAR, 360° horizontal × ±15° vertical FOV) provides point clouds at ~5-20 Hz. In ROS this is typically published as a sensor_msgs/PointCloud2.
- **Cameras:** It produces images (e.g. KITTI's 1.4MP cameras at 1382×512 px, 10 Hz) as sensor_msgs/Image. Each camera provides image streams (e.g. raw Bayer or compressed RGB) at its frame rate (often 30–60 Hz).
- **IMU:** Typically 3-axis accelerometer and gyro at high rate (100–1000 Hz). Outputs linear acceleration (m/s²) and angular velocity (rad/s) in the IMU frame, plus optional magnetometer.
- **GPS:** Provides global position (latitude, longitude, altitude) and velocity via NMEA or custom binary.

Data Acquisition and Pipeline

- **Public Datasets:** eg: KITTI Dataset([Link](#))
- **ROS bags files:** /velodyne_packets, /camera/image_raw, /imu/data, /gps/fix.

Output Interface (Localization)

The localization algorithm under test is treated as a black box that outputs a *pose estimate*.

The schema includes: **position (x,y,z)**, **orientation (e.g. quaternion or yaw/pitch/roll)**, velocity (optional), and an associated 6×6 covariance (or diagonal) matrix. Each output has a timestamp and a coordinate frame ID (e.g. "map" or "odom"). The Gold Standard trajectory uses the same schema for direct error computation. Also, we will visualize and compare against the gold standard in a 2D plot for comparing.

Perturbation YAML Schema

We propose a YAML configuration to specify perturbations per sensor. For example:

```
perturbations:
  lidar:
    noise_std: 0.02      # additive Gaussian range noise (meters)
    dropout_fraction: 0.1 # fraction of points to drop randomly
    intensity_scale: 0.8 # multiply reflectivity by <1
    beam_occlusion:      # simulate tree occlusion
      enabled: true
      region: [[-1,1], [10,20]] # block points in 1<x<10m, 10<z<20m
  camera:
    gaussian_noise: 0.01 # pixel noise (normalized 0-1)
    brightness_factor: 0.5 # scale pixel intensities (low-light)
    motion_blur_kernel: 5 # kernel size for linear blur
    exposure_shift: -2.0 # stops change (negative = darker)
    raindrop_overlay:    # add random raindrop decals
      enabled: true
      count: 30
    lens_flare:          # simulate sun glare
      enabled: true
      intensity: 0.3
  imu:
    gyro_bias: [0.001,0.0,0.002] # rad/s biases on axes
    accel_noise: 0.05             # m/s^2 noise std
    accel_bias_drift: 0.0001      # m/s^3 bias random walk
  gps:
    drop_probability: 0.1 # fraction of seconds with no fix
    multipath:          # add sine-wave error to lat/lon
      enabled: true
      amplitude: 3.0     # meters
      frequency: 0.5     # Hz
  global:
    time_offset: 0.05     # clock skew (seconds)
```

Real-World Scenario Simulations

Low-Light Conditions

Low-light conditions occur during nighttime operation, tunnels, underground parking, and poorly illuminated environments. Reduced illumination decreases image contrast and feature visibility, making visual feature extraction and tracking more difficult. The Velodyne VLP-16 is an active infrared sensor and is largely unaffected by ambient lighting conditions.

Camera: Data Modification: Image brightness will be reduced using intensity scaling:

I' is the modified image, and α is the brightness reduction factor ($0 < \alpha < 1$). To emulate increased sensor noise commonly observed in low-light conditions, Gaussian noise will be added:

$$I'' = I' + N(0, \sigma^2)$$

where $N(0, \sigma^2)$ is Gaussian noise and σ controls noise intensity.

LiDAR: No LiDAR points will be modified.

YAML Example:

```
camera:
  low_light:
    brightness_factor: 0.4
    noise_std: 10

lidar:
  low_light:
    enabled: false
```

Glare and High-Contrast Conditions

Glare and high-contrast conditions occur when strong light sources such as direct sunlight, vehicle headlights, or reflective surfaces enter the camera field of view. This can cause image saturation, reduced contrast, and loss of visual features. For LiDAR, highly reflective surfaces such as wet roads, glass, or metallic objects may generate abnormal returns.

Camera: Pixels within high-intensity regions of the image ($I > T$), where T is a brightness threshold, will be selected. Selected pixels will be amplified and clipped to simulate sensor saturation

$$I' = \min(\alpha I, 255)$$

where I is the original pixel intensity, I' is the modified pixel intensity, and $\alpha > 1$ controls glare severity.

LiDAR: Points located within a predefined reflective region of the point cloud will be selected using spatial bounds. A percentage of selected points will be removed to simulate missing returns, while additional ghost points will be generated by perturbing existing points:

$$P' = P + N(0, \sigma^2)$$

where P is an original point in the reflective region, P' is the generated ghost point, and $N(0, \sigma^2)$ is Gaussian positional noise.

YAML Example:

```
camera:
  glare:
    intensity_threshold: 220
    brightness_scale: 1.5

lidar:
  reflective_region:
    x_min: 5
    x_max: 15
    y_min: -3
    y_max: 3
    point_drop_ratio: 0.2
    ghost_point_ratio: 0.1
    noise_std: 0.2
```

Tunnel / Illumination Transitions

Tunnel and illumination transition scenarios occur when a vehicle moves rapidly between bright and dark environments, such as entering or exiting tunnels, parking structures, or heavily shaded roads. Camera sensors require time to adjust exposure settings, causing temporary overexposed or underexposed frames. The VLP-16 LiDAR is largely unaffected by ambient lighting changes and continues to provide stable range measurements.

Camera: All image frames within a specified transition interval will be selected, corresponding to tunnel entry or exit. Image brightness will be gradually adjusted across consecutive frames to simulate camera auto-exposure adaptation:

$$I_t = \alpha_t I$$

where I is the original image, I_t is the modified image at time t , and α_t varies linearly from an initial brightness factor to a final brightness factor over the transition period.

LiDAR: No LiDAR points will be modified.

YAML Example:

```
camera:
  tunnel_transition:
    start_time: 120
    end_time: 135
    brightness_start: 0.3
    brightness_end: 1.0

lidar:
  tunnel_transition:
    enabled: false
```

Rain (Water on Lens and in Air)

Rain affects both camera and LiDAR sensors. Water droplets on the camera lens create blur and partial occlusions, while rain streaks reduce image clarity and contrast. For the Velodyne VLP-16, raindrops scatter infrared laser pulses, causing missing returns, reduced intensity values, and occasional ghost measurements.

Camera: All image frames within the configured rain interval will be selected. Synthetic rain streaks will be overlaid on the image and contrast reduced:

$$I' = (1 - \alpha)I + \alpha R$$

where I is the original image, I' is the modified image, R is the generated rain mask, and α controls rain intensity.

LiDAR: All points within the point cloud will be processed. A percentage of points will be randomly removed to simulate occlusion by raindrops:

$$P_{remain} = (1 - r_d)P$$

where P is the original set of points and r_d is the point drop ratio.

Additional ghost points will be generated within a specified distance from the sensor:

$$P' = P + N(0, \sigma^2)$$

where P is a selected point, P' is the generated ghost point, and $N(0, \sigma^2)$ is Gaussian positional noise.

Point intensity values will also be attenuated:

$$I' = \beta I$$

where I is the original intensity, I' is the modified intensity, and $\beta < 1$ controls attenuation severity.

YAML Example:

```
camera:
  rain:
    intensity: 0.4
    streak_length: 15
    contrast_scale: 0.8
    droplet_count: 20
    droplet_blur_kernel: 11

lidar:
  rain:
    point_drop_ratio: 0.2
    ghost_point_ratio: 0.05
    ghost_range_max: 5
    intensity_scale: 0.8
    noise_std: 0.1
```

Fog, Dust and Haze

Fog, haze, and smoke reduce visibility by scattering and absorbing light. For cameras, distant objects appear faded, low-contrast, and less distinguishable. For the Velodyne VLP-16, suspended particles attenuate infrared laser pulses, reducing return intensity, decreasing effective sensing range, and generating occasional backscatter measurements.

Camera: All image frames within the configured fog, haze, or smoke interval will be selected. A visibility-reducing layer will be applied to the entire image using atmospheric attenuation:

$$I' = Ie^{-\beta}$$

where I is the original image, I' is the modified image, and β is the attenuation coefficient controlling fog or smoke density.

LiDAR: All points whose distance from the sensor exceeds a specified visibility distance will be selected. The intensity of selected points will be attenuated:

$$R' = \alpha R$$

where R is the original intensity, R' is the modified intensity, and $\alpha < 1$ controls attenuation severity.

Additionally, a percentage of distant points will be removed to simulate lost returns, and ghost points will be inserted within the visibility range to emulate particle backscatter.

YAML Example:

```
camera:
  visibility_reduction:
    density: 0.5
    overlay_opacity: 0.3

lidar:
  visibility_reduction:
    visibility_distance: 30
    intensity_scale: 0.7
    point_drop_ratio: 0.15
    backscatter_ratio: 0.05
```

Foliage and Occlusions

Foliage and occlusions occur when tree branches, leaves, bushes, parked vehicles, or other objects partially block the sensor field of view. For cameras, these objects hide portions of the scene and reduce the number of visible features. For the Velodyne VLP-16, occluding objects block laser beams, resulting in missing points within specific viewing directions.

Camera: A predefined image region corresponding to the occluding object will be selected using image coordinates. Pixels within the selected region will be replaced with a synthetic foliage mask or partially darkened to simulate visual obstruction:

$$I'(x, y) = (1 - \alpha)I(x, y) + \alpha M(x, y)$$

where $I(x, y)$ is the original pixel value, $M(x, y)$ is the occlusion mask, and α controls occlusion opacity.

LiDAR: Points whose azimuth angles fall within the sector blocked by the foliage or obstacle will be selected. A percentage of selected points will be removed to simulate laser beam blockage:

$$P_{remain} = (1 - r_d)P$$

where P is the original set of points in the occluded sector and r_d is the point drop ratio.

YAML Example:

```
camera:
  foliage_occlusion:
    x_min: 200
    x_max: 500
    y_min: 100
    y_max: 400
    opacity: 0.8

lidar:
  foliage_occlusion:
    azimuth_start: 40
    azimuth_end: 70
    point_drop_ratio: 0.8
```

Partial Sensor Failure

Partial sensor failure occurs when a sensor continues operating but only a portion of its data becomes unavailable or corrupted. Examples include dropped camera frames, missing LiDAR packets, frozen IMU measurements, or intermittent GPS signal loss. These failures reduce the amount of information available to the localization system and can significantly impact pose estimation.

Camera: Image frames within a specified time interval will be selected. A percentage of frames will be replaced by the previous frame to simulate a frozen camera stream:

$$I'_t = I_{t-1}$$

where I'_t is the modified frame and I_{t-1} is the previous frame.

LiDAR: LiDAR scans within the specified failure interval will be selected. A percentage of complete scans will be removed to simulate packet loss or communication failures:

$$S_{remain} = (1 - r_d)S$$

where S is the original set of scans and r_d is the scan drop ratio.

IMU: IMU measurements within the specified failure interval will be selected. Selected measurements will be frozen at their previous value:

$$M'_t = M_{t-1}$$

where M_t represents accelerometer and gyroscope readings.

GPS: GPS messages within the specified failure interval will be selected. A percentage of GPS messages will be removed to simulate intermittent signal loss:

$$G_{remain} = (1 - r_d)G$$

where G is the original set of GPS measurements and r_d is the message drop ratio.

YAML Example:

```
camera:
  partial_failure:
    start_time: 120
    end_time: 140
    freeze_ratio: 0.3

lidar:
  partial_failure:
    start_time: 120
    end_time: 140
    scan_drop_ratio: 0.3
```

```
imu:
  partial_failure:
    start_time: 120
    end_time: 140
    freeze_ratio: 0.3

gps:
  partial_failure:
    start_time: 120
    end_time: 140
    message_drop_ratio: 0.3
```

Vibration / Shock

Vibration and shock occur due to rough terrain, potholes, speed bumps, engine vibrations, or sudden impacts. These disturbances cause camera image blur and jitter, LiDAR scan misalignment due to sensor movement, and noisy IMU measurements. Such effects can degrade feature tracking, point cloud registration, and motion estimation.

Camera: All image frames within the specified vibration interval will be selected. Motion blur will be applied to the entire image using a blur kernel, and each frame will be shifted by a small random rotation to simulate camera shake:

$$I' = B(I, k)$$

where I is the original image, I' is the blurred image, and k is the blur kernel size.

LiDAR: All points in each LiDAR scan will be selected. A small random pose perturbation will be applied to every point cloud frame:

$$P' = T(\Delta x, \Delta y, \Delta z, \Delta \psi)P$$

where P is the original point, P' is the transformed point, and T is a random translation and rotation representing sensor vibration.

IMU: All IMU measurements within the specified interval will be selected. High-frequency Gaussian noise will be added to accelerometer and gyroscope measurements:

$$M' = M + N(0, \sigma^2)$$

where M is the original IMU measurement, M' is the modified measurement, and σ controls vibration severity.

YAML Example:

```
camera:
  vibration:
    blur_kernel: 9
    rotation_std_deg: 1.0

lidar:
  vibration:
    position_sigma: 0.05
    yaw_sigma_deg: 0.5

imu:
  vibration:
    accel_noise_std: 0.2
    gyro_noise_std: 0.05
```

IMU Bias Drift and Temperature Effects

IMU bias drift occurs when accelerometer and gyroscope measurements slowly deviate from their true values over time due to sensor aging, temperature changes, or hardware imperfections.

IMU: All IMU measurements within the specified time interval will be selected. A time-varying bias will be added to accelerometer and gyroscope measurements:

$$M' = M + bt$$

where M is the original IMU measurement, M' is the modified measurement, b is the drift rate, and t is the elapsed time from the start of the fault injection.

To simulate temperature effects, a scale-factor error may also be applied:

$$M' = (1 + \epsilon)M + bt$$

where ϵ is the temperature-induced scale factor error.

YAML Example:

```
imu:
  bias_drift:
    gyro_drift_rate: 0.001
    accel_drift_rate: 0.01
    scale_factor_error: 0.02
```

Evaluation Methodology

To assess the impact of each perturbation, we compare the test output to the Gold Standard trajectory (which is error-free). Align both trajectories by timestamp (e.g. interpolate the Gold Standard to test output times). Compute per-frame errors:

- **Longitudinal error** (along the nominal travel direction) and **lateral error** (perpendicular to travel), both in meters. For unstructured trajectories, one can project the error vector into Frenet coordinates relative to motion.
- **Yaw** (heading) error in degrees or radians.

From these, compute summary metrics: **RMSE** (root-mean-square) of longitudinal, lateral, vertical, and yaw errors, plus mean/median and max error. Also record time-to-failure: the time until error exceeds a safety threshold. Compute drift rate (m/s of error growth per second of run).

- **Analysis per event:** For each injected event (e.g. rain start), measure error just before, during, and after to isolate impact. Aggregate over multiple runs (stochastic seeds) and perform statistical tests (e.g. t-test or nonparametric tests) to see if errors under perturbation are significantly higher.
- **Visualizations:** Plot error vs. time, and error vs. perturbation intensity (e.g. plot lateral error vs. percentage of LiDAR points dropped). Bar charts or boxplots can summarize errors for each scenario. A scatter or line chart of error magnitude vs. noise magnitude (or vs. visibility distance) helps identify sensitivity. The YAML parameters can be swept to generate such charts.

Profiling & Runtime Measurement

Finally, we will profile the computational performance of the localization stack under varying sensor loads. The plan is to measure:

- **CPU/GPU Usage:** Monitor processor utilization when running the algorithm with full-rate inputs. Tools like `htop`, `ros2 run rqt_graph`, or ROS2 tracing (`ros2 tracing`) can log CPU time per node.
- **Throughput & Latency:** Measure how many input frames per second the stack can process, and the end-to-end latency (from sensor timestamp to pose output). For instance, run the localization node on the KITTI dataset in real time (or faster/slower) and record average cycle time.
- **Scaling Tests:** Vary input rate or point-cloud density to see how runtime scales (e.g. downsample LiDAR).

We will record these metrics on our target hardware (e.g. the robot's CPU). The goal is to ensure the localization runs in real time with all sensors active.